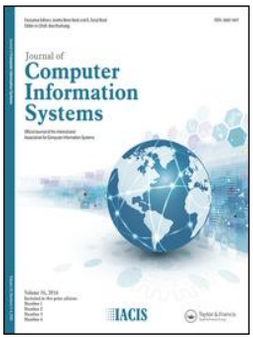

Journal of Computer Information Systems

ISSN: 0887-4417 (Print) 2380-2057 (Online) Journal homepage: www.tandfonline.com/journals/ucis20

Assessing Self-Organization in Agile Software Development Teams

Adarsh Kumar Kakar

To cite this article: Adarsh Kumar Kakar (2017) Assessing Self-Organization in Agile Software Development Teams, Journal of Computer Information Systems, 57:3, 208-217, DOI: 10.1080/07362994.2016.1184002

To link to this article: https://doi.org/10.1080/07362994.2016.1184002

Published online: 30 Aug 2016.

JOURNAL OF COMPUTER INFORMATION SYSTEMS
2017, VOL. 57, NO. 3, 208–217
http://dx.doi.org/10.1080/07362994.2016.1184002

Taylor & Francis
Taylor & Francis Group

Assessing Self-Organization in Agile Software Development Teams

Adarsh Kumar Kakar

Alabama State University, Montgomery, AL, USA

ABSTRACT

Self-organization, based on the Socio-Technical Design (STS) work design principles, is considered a hallmark of agile software development (SD) teams and an antecedent of motivation and innovation at work. However, while self-organization is considered a salient success factor in agile SD, past research has shown that there are many hurdles in self-organizing. Yet, the actual level of self-organization practiced in agile teams has not been investigated. In this study we develop a measure to do so using the nine principles of STS work design by Cherns [15]. We found that while the level of self-organization varies across agile projects, on each of the nine dimensions the level of self-organization in agile teams was found to be significantly higher than those using plan-driven methods. Furthermore, self-organization was found to positively affect the motivation and innovativeness of SD teams.

KEYWORDS

Self-organizing teams; socio-technical systems design; Cherns principles; agile methods; plan-driven methods

Introduction

In the plan-driven methods of software development (SD), project managers, under supervision of senior management, are charged with setting project goals, allocating tasks to team members, evaluating their performance and coordinating with the customer [45]. Furthermore, in accordance with the Tayloristic principles, team members in plan-driven methods execute their assigned tasks within the confines of their specialized roles as designers, business analysts, programmers, testers and implementers and by adhering to the projects' defined process [64]. By contrast, agile methods adopt the Socio-Technical Design (STS) work design by deploying self-organizing teams. Agile teams set and accomplish their own goals through participation among team members, which includes a customer representative. Agile team members are multi-skilled, have autonomy in the way they perform tasks and repeatedly reorganize by allocating work among themselves based on changing circumstances [74].

It is generally acknowledged that systems developed by adopting an STS approach deliver better value and provide solutions that are more acceptable to end users [8]. Self-organization stimulates greater team member involvement and participation, resulting in higher commitment and motivation. This leads to higher employee productivity and product quality [29]. Employees care more about their tasks and work cooperatively with others to achieve team goals. By bringing decision making to the level of people who actually perform work teams are able to quickly respond to changing environment [29, 95]. Further, team members of self-organizing groups demonstrate greater creativity and problem-solving skills [29, 95, 68].

Keeping these salutary psychosocial and work outcomes of self-organizing teams in view, it is relevant to investigate whether agile teams actually exhibit high self-organization. Self-organization is not easy to achieve in practice. There are many human and organizational hurdles that may not be conducive to its implementation. Self-transcendence, in which teams evaluate themselves continuously to devise better ways of achieving their own goals, and a key requirement of self-organization, requires a change in the mindset of people, which is not easy to achieve [69]. Further, senior management may be unwilling to give up control over team workings. Several aspects of the organization such as its culture, structure and management practices need to be changed to transition to self-organization [68].

Keeping this context in view, the question the study endeavors to answer is "What is the current state of practice of self-organization in agile software development teams?" Currently there are gaps in the literature. First, no approach or instrument has yet been designed to measure and compare self-organization between teams. Second, although conceptually the difference in adoption of self-organization in agile versus plan-driven methods has been discussed previously in the literature, the levels of self-organization between these two major paradigms of SD have not been objectively compared.

To achieve these goals we first develop a deeper understanding of the STS work design perspective by tracing the historical issues that lead to its development. We identify the benefits of self-organization and barriers to its implementation in the context of SD. We then develop a measure for assessing the level of self-organization in teams using the nine STS work design principles defined by Cherns [15] as a theoretical framework. Cherns' [15] principles provide a systematic and clear overview of the major standpoints of STS [47] on work design issues. These principles and the measure derived from these principles enable us to compare the level of self-organization in the two SD paradigms on a point-by-point basis. Finally, we assess the impacts of self-organization on innovation and motivation in SD teams. Motivation of

CONTACT Adarsh Kumar Kakar akakar@alasu.edu Department of Computer Information Systems, Alabama State University, 915 South Jackson Street, Montgomery, AL 63104, USA.

team members is considered a "particularly pernicious people problem" [10]. Further, creativity and innovation are widely acknowledged as core components of software development [12, 20, 56, 72, 89, 97]. However, although a review of the literature suggests that self-organization will positively impact both motivation and innovation, yet these salutary and key impacts of self-organization on SD have not been empirically validated thus far. With the development of a measure for self-organization, we hope to make this assessment in the study.

Literature review

Historical origins

Frederick Winslow Taylor [96] introduced Scientific Management with the aim of controlling every work activity, from the simplest to the most complicated. He applied to workers the ideas Whitney (see [66]) earlier used for making interchangeable parts. Taylor analyzed tasks into their minutest details and arrived at a standardized process; the one best way to do the job [52], just as Eli Whitney analyzed, is to musket into its smallest parts and make a machine to manufacture each part [66]. Together the ideas of Whitney, Taylor and Ford ushered in the era of mass production. However, while mass production resulted in an improvement in the standard of living of society, it had deleterious psychological consequences for the workers. Repetitive jobs were found to be boring, tiring, dissatisfying and potentially damaging to mental health [31, 101].

Socio-Technical Systems (STS) design was the first alternative work design paradigm that challenged the scientific management principles of Taylor. Since the experiments of the Tavistock Institute in the early 1950s [99], the work design concepts of STS have become a major influence and elements of STS have now been implemented in firms around the world [17]. While the Tayloristic practices and principles focused on the production process, the STS practices and concepts, on the other hand, were "human-centered" [3]. STS viewed people as a resource to be developed [98]. Business Week (September 29, 1986), predicted that this movement toward STS is "an immensely important trend, one that is producing a new model of job design and work relations that will shape the workplace well into the twenty-first century."

STS work design advocates self-organizing teams. It implies that technology such as machines and work tools and the organization outside the team should not be the controlling factor [71]. Guzzo and Dickson [37] describe such teams "as teams of employees who typically perform highly related or interdependent jobs, who are identified and identifiable as a social unit in an organization, and who are given significant authority and responsibility for many aspects of their work, such as planning, scheduling, assigning tasks to members, and making decisions with economic consequences" [69]. They differ from semi-autonomous groups by having both the knowledge and the authority to set goals and accomplish them [71].

A parallel to these developments in the domain of manufacturing work design can be found in SD. Earlier, Frederick Taylor's ideas of scientific management had a huge influence on SD. As applied to software development, scientific management led to the development of factory-like concepts. R. W. Bemer [11] suggested adoption of standardized tools to reduce variability in programmer productivity and keeping a database of historical records for management control. M . D. McIlroy [63] emphasized systematic reusability of code for enhancing productivity. Attempts were made to introduce statistical control in software engineering [48]. Efficiency of SD processes was measured through the use of control charts. Taylorist approaches such as the waterfall model [86] and its variants gained popularity. These methods promoted strong conformance to plan through upfront requirements gathering and systems design, programming standards, code inspections and productivity metrics. They encouraged division of labor, leading to specialized roles of business analysts, system architects, programmers and testers [64].

However, these process-centric plan-driven approaches to SD lost ground to people-focused agile approaches [2]. Organizations realized that while process-focused approaches worked in stable environments, under uncertain conditions managers planning, assigning and controlling tasks of software developers may not work. Further, under conditions of rapidly evolving customer needs, the Taylorist approach of first specifying customer requirements fully before working on them did not seem appropriate. With increasing problem complexity, changing scope and requirements, and evolving technologies, developers, over time, came to realize that an alternative approach is needed. This alternative came in the form the Agile Manifesto [1], a people-focused approach to SD, which suggested among other things deployment of self-organizing teams for SD [74].

Benefits and challenges of self-organized teaming

Self-organizing teams provide advantages over Tayloristic teams. Bringing decision making to the level of teams enables them to respond quickly and appropriately to changing environments [29, 95]. The positive psychosocial effects of autonomy in work environments are well documented in the literature. Autonomy not only provides the employees opportunity to master new responsibilities and acquire new skills but also boosts self-efficacy by increasing the controllability of a task [35]. Further with increased autonomy employees develop confidence and capabilities for carrying out a wider range of responsibilities and tasks [80]. Armed with this higher role-breadth self-efficacy employees proactively seek and strive to achieve more challenging goals [82].

The motivational effects of autonomy are supported by the observed empirical findings [10, 81, 85]. Kramer [77] observed that restricting autonomy has a deleterious effect on performance as formal monitoring and control systems create the impression that the management does not trust them. Autonomy prevents job strain by enabling employees to directly reduce stressful work aspects [32]. Other positive impacts of increased autonomy include cognitive development [54], increased self-efficacy [13, 80, 103], more responsibility for external coordination with those in other departments [6], creativity and innovation, the development of more proactive

role orientations [82] and greater use of personal initiative and engagement in accomplishing tasks [33].

Organizations thus deploy self-organizing teams to improve productivity and quality and to reduce costs [68]. Such teams are observed to have lower absenteeism, higher employee satisfaction and lower turnover [68]. As a result agile methods have increasingly replaced the erstwhile plan-driven approaches of SD. Heavyweight methods such as Structured Systems Analysis and Design Methodology [28], Information Engineering [59], Unified Software Development Process [50] and OPEN [36] lost out to [16] light-weight methods that cope well under conditions of high change and uncertainty such as Extreme programming [34], Scrum [91], Crystal methodologies [18], Dynamic Software Development Method (DSDM) [93], Lean Software development [84] and Feature Driven Development (FDD) [79].

However, there are numerous barriers to practicing self-organization [68]. Most employees are familiar with the command and control work environments where individual goals assume more priority over team goals. Developers worked on accomplishing their assigned tasks efficiently in hope of reward and recognition. Adjusting to a self-organizing environment where collective responsibility and group goals are a norm is not easy [45]. Implementing shared leadership was difficult. Team members need to genuinely commit to formulating and achieving team goals [68]. Redundancy of functions is a requirement for achieving flexibility in self-organizing teams. This requires employees to be able to perform multiple functions [49]. However, while multiskilled team members may be able to perform several roles of programmers, testers, designers and requirements analysis with flexibility based on work demands, recruiting and training such employees is difficult and expensive [100].

Further, self-organizing teams require a conducive organizational environment to thrive [74]. A misalignment between team and organization structure can be counterproductive for both. Senior managers in optimizing Tayloristic organizations may be unwilling to give up control [19]. Self-organizing teams may not have full control over their resources as employees may get shifted between projects depending on organizational priorities [68]. Therefore, although agile teams and organizations may profess to practicing self-management, the reality may be quite different.

To find out the true level of self-organization practiced in agile teams we use the nine STS design principles defined by Cherns [15] as a benchmark. The Agile manifesto and its 12 principles are not suitable for the purpose. In its current form the Agile manifesto neither represents theory nor provides the tools to assess the level of self-organizations in teams. On the other hand, STS is a theoretically well-developed concept [75]. We therefore found the nine STS design principles defined by Cherns [15] particularly useful as a theoretical framework for the purpose of our investigations. It is a framework that is often used to provide a systematic and clear overview of the major standpoints of STS on work design issues [47].

Hypotheses development

In this section we discuss the level of self-organization practiced in agile and plan-driven SD teams with respect to each of the nine Cherns [15] principles and formulate hypotheses for empirical validation.

Minimum critical specification

According to Cherns [15], the principle of minimum critical specification is defining as little as possible how tasks should be performed. STS favors the principle of self-management [42, 70] and allowing as much autonomy in terms of work pace and work methods as possible for performing tasks [38]. In plan-driven methods the work is planned in advance by managers and assigned to employees by the immediate supervisor. The allocation of work specifies "not only what is to be done but how it is to be done and the time allowed for doing it" [14]. Process controls such as standards and procedures are enforced to reduce variability of work outcomes [61]. By contrast, agile methods rely on teamwork, as opposed to individual role assignment that characterizes plan-driven SD [74]. Team members organize their work tasks in a way that gave them common ownership of the work product and control over how it was achieved [26].

Hypothesis 1: Team members of SD projects using agile methods have significantly more autonomy on how they perform tasks than team members of SD projects using plan-driven methods.

Boundaries

Boundaries between groups are strongly emphasized in STS [42]. Boundaries decouple the production process, making teams responsible for "whole tasks" and increasing interdependence. The downside is that the teams may become too inwardly oriented, assert their own localized interest and exhibit groupthink [62]. Agile teams work within team and physical boundaries. It is therefore not surprising that agile SD projects are seen as restricted to small colocated teams. For example, unlike plan-driven methods agile methods do not promote reusable artifacts. Developing code for future use is considered as excess inventory. Agile methods follow the dictum – "don't do anything today that you can postpone to tomorrow" [51]. The story cards are detailed only for current iteration. Developing common parts and modularization is critical to achieving flexibility and overall efficiency. Yet agile methods preclude developing reusable codes even though it could provide long-term benefits across development projects [100].

Hypothesis 2: Team members of SD projects using agile methods are significantly less concerned with issues outside their project domain than team members of SD projects using plan-driven methods.

Multifunctionality

This Cherns [15] principle states that workers should possess a variety of relevant skills and should be capable of performing a diverse range of tasks. Multiskilled workers create reserve capacity and open up possibilities for integrating tasks and creating meaningful work. Plan-driven methods adopt the Taylorist principle of specialization and division of labor, leading to specialized roles of business analysts,

system architects, programmers and testers [61]. In agile methods tasks are not specialized to the degree of plan-driven methods. Redundancy in functions is encouraged to enable flexibility during absences and shortages of personnel [69]. Programmers may perform multiple tasks such as designing, coding, testing and even requirements elicitation.

Hypothesis 3: Team members of SD projects using agile methods possess significantly greater degree of skill variety for performing tasks than team members of SD projects using plan-driven methods.

Support congruence

According to this Cherns [15] principle, human resource systems such as training, assessment, rewards and promotion should coherently support the activities of team members. Taylorist and agile methods differ on their approach to support congruence. For example, although Taylorist and STS both support training, their emphases are different. Both acknowledge that only knowledgeable and skilled employees can meet the organizational needs. However, STS supports a broad range of training, including in areas of supervisory and analytical skills to perform and improve jobs. On the other hand, Taylorist approaches focus on skill enhancement of workers to perform their specialized roles more effectively.

Hypothesis 4: Team members of SD projects using agile methods undergo significantly more variety of coaching and training than team members of SD projects using plan-driven methods.

Feedback

This Cherns [15] principle states that the feedback about tasks should be made available to the employees who perform them. Feedback provides the employees information on the effectiveness of their efforts [38]. STS realizes that feedback provides information that enables the employee to make the necessary corrections. It enhances employee autonomy by providing them with learning opportunities and allowing them to make their own decisions. In Taylorist approaches feedback information is primarily used to detect deviation from standards and provide transparency to process issues. It is used for surveillance and monitoring of the employee activities. Accordingly the methods of feedback in agile and plan-driven methods also vary. Agile methods use daily stand-up meeting, sprint reviews and project retrospectives. Plan-driven methods use formal productivity reports to highlight discrepancy between the production targets, and performance levels and quality problems are discussed in regular periodic team review meetings.

Hypothesis 5: Team members of SD projects using agile methods receive significantly greater feedback on their performance than team members of SD projects using plan-driven methods.

Incompletion

The principle of incompletion states that the design is an ongoing process. In STS, design should be part of the daily work and is an outcome of a continuous cycle of learning and experimentation. This principle finds its full expression in agile methods of SD. For example, agile projects starting with the smallest set of the most critical customer requirements to initiate a project [73]. They work on the principle of developing working products in multiple iterations. Each iteration acts as a cycle of discovery where users review actual working product instead of paper reviews or review of prototypes in plan-driven methods. These working products become the basis for further discussions and the team works toward delivering the business solution using the latest input from customers, users and other stakeholders. As the solution emerges through working products, the application design, architecture and business priorities are continuously evaluated and refactored.

By contrast plan-driven approaches focus on complete requirement specification, upfront planning, detailed design and sequential development phases [74]. However, according to agile methods software cannot be fully specified up-front [44] as business requirements and technologies change rapidly during the course of an SD project. The true requirements should be allowed to emerge over time because what the customer initially thought they wanted gets refined as software develops.

Hypothesis 6: Team members of SD projects using agile methods develop the software in significantly more iterations compared to team members of SD projects using plan-driven methods.

Compatibility

In the design process STS recommends a fit between the approach used and the desired outcome. A design by experts thrust on the team will not lead to participation and ownership by the team members. The most effective designs are likely to come from those whose job it is to implement them [27]. This approach is adopted by agile methods where the coders are an integral part of the design process. There are no specialized roles for designers, system analysts and architects. In stark contrast, the Tayloristic principles emphasize design by experts. Accordingly in plan-driven SD designers and architects play the crucial role in the design process and the coders are assigned the responsibility of implementing them.

Hypothesis 7: Team members of SD projects using agile methods have significantly greater participation in the design process than team members of SD projects using plan-driven methods.

Sociotechnical criterion

This STS principle stipulates that variations should be controlled at the point of their origin. The assumption is that the employee responsible for the problem is also in the best

position to correct it [92]. By contrast the Taylorist approach fixes the responsibility of quality on the quality control department. In line with the Taylorist approach plan-driven methods have separate quality and testing teams. Quality processes such as end-of-line testing, statistical process control and formal inspection and verification are the norms. On the other hand, agile approaches do not have a separate quality control and testing teams. Errors are detected at source through test-driven development and frequent code integration.

Hypothesis 8: Team members of SD projects using agile methods identify and resolve significantly more problems themselves than team members of SD projects using plan-driven methods.

Human values

The principle of "Human Values" prescribes that the organization should be sincerely concerned with the human needs and improving the Quality of Working Life (QWL) including work content, reward and compensation systems, employee relations and working conditions. While both Taylorism and STS acknowledge the importance of addressing human needs by taking care of working conditions and compensation systems, the Taylorist approach of standardized work and narrow tasks to be performed in short cycles does not adequately address work content issues. On the other hand, STS pays special attention to work content through job enrichment. It believes that employees have the ability to participate in management and can be empowered to make their own decisions. By maximizing individual and group autonomy STS seeks to humanize the workplace [53]. In accordance with these differences, plan-driven methods deploy hierarchical teams with command control structures as against self-managing teams of agile methods.

Hypothesis 9: Team members of SD projects using agile methods have a significantly higher perception that their personal and professional needs are being taken care of than team members of SD projects using plan-driven methods.

The differences between the practices of agile and plan-driven methods of SD are summarized in Table 1. Many of the agile practices in Table 1 are compiled from Scrum and eXtreme Programming (XP), two of the most widely deployed agile methods [4]. Their practices are listed together because while the project management framework is described by Scrum, XP describes the development practices [41].

According to the Self-Determination Theory (SDT) [87], there are three basic psychological needs that motivate people: the need for autonomy [23], competence [102] and relatedness [7]. The need for autonomy is the desire to have control over one's own actions [23]; the need for competence is the desire to perform challenging tasks and accomplish challenging goals [102]; and the need for relatedness is the need for interpersonal connection and feelings of a sense of closeness with others [7].

Table 1. Differences between agile and plan-driven methods from the STS perspective.

Cherns [15] principles	Plan-driven methods	Agile methods
Minimum Critical Specification	Requirements sign-off by the customer; detailed design; defined process; direct supervision and control [83]	Minimum critical specification; emergence; self- management; mutual adjustment; concertive control [74].
Boundaries	Organization	Team [62]
Multi-functionality	Specialization; specialized roles: designers, programmers, testers [64]	Multiple skills; recruiting and developing multiskilled employees [69]
Support Congruence	Skill-specific training	Broadly oriented training
Feedback	Aimed at transparency; used for control; inspections, formal reviews	Aimed at enhancing autonomy; double-loop learning; daily stand-up meetings, retrospectives; daily stand-up meetings, face-to-face communication promotes tacit knowledge sharing; paired programming; refactoring and retrospectives [9, 90] Plan for current iteration only; focus on people and interactions rather than on process and documentation [1, 9, 90]
Incompletion	Upfront planning, defined process and extensive documentation [83]	
Compatibility	Design by experts; automation of development processes, e.g. CASE (Computer Aided Software Engineering) Tools [48]; hierarchical team structure; reviews, audits and inspection [67, 83]	Self-design; simple tools, focus on people interaction [1, 9, 90]
Socio-Technical Criterion	Limiting procedures; end-of-line testing [48], statistical process control [30]	Employees can change norms; refactoring and retrospectives [9, 90]
Human Values	Employee relations; work environment	Quality of life; focus on work content

We expect the self-organizing teams to fulfill all these three needs of employees better than teams working in command and control environments. Self-organizing agile teams provide greater autonomy to the ways employees perform their task compared to the process-focused plan-driven environments. By requiring employees to develop multiple skills and role-breadth efficacy, self-organizing teams are better able to satisfy the need for competence. The participative approach to problem solving, setting group goals and helpful feedback behaviors promote relatedness. As a result we will expect agile teams to display higher motivation than plan-driven teams.

Hypothesis 10: Self-organization is positively related to employee motivation.

Hypothesis 11: Team members working in SD projects using agile methods have a significantly higher work motivation than team members of SD projects using plan-driven methods.

Self-organization is considered the first principle for designing collaborative and innovative teams [65]. Past research findings suggest that self-organization is salient for creativity and innovation in projects [46, 94]. Creativity and innovation thrive when employees feel free to experiment and are not constrained by processes in the way they accomplish tasks [76]. Autonomous teams provide this environment of freedom to deviate from and tailor work processes [5, 60]. Self-organizing agile teams are therefore best positioned to foster creativity and innovation in SD [43].

Hypothesis 12: Self-organization is positively related to team innovation.

Hypothesis 13: SD projects using agile methods will demonstrate higher innovation than team members of SD projects using plan-driven methods.

Method

Study setting and design

To test our hypotheses we conducted a survey with development team members of 34 software projects. The developers were employees of the university's industry partners and graduate students of the university who had to complete a real-life software project with the industry partners, which included 18 companies, with three of them in the Fortune 500 list. The type of projects included 14 which the industry partners characterized as Waterfall method, four as V-method, nine as Extreme programming, three as Scrum, one as Crystal methodologies, one as DSDM, one as FDD and one as Lean Software Development Method (LSDM).

The capstone projects enable students to work on a real-life project and provide them with job opportunities. The university has a very high placement rate and many of the students who work on the capstone projects are employed by the industry partners. The study was completed over a 4-year period involving 342 SD team members including 167 students. The subjects answered a pen and pencil questionnaire-based survey at the end of completion of the projects and represented the response from 84 % of developers who participated in the 34 development projects.

Subjects and procedure

The subjects were between 21 and 39 years old, 194 males and 148 females who worked on SD projects involving between 6 and 16 team members. The average age of the subjects was 28.4 years, average experience on real-life SD projects was 6.3 years and the average number of team members working on the projects was 10. Subjects answered a paper and pencil-based survey that captured data on: the self-organization measure for each of the nine Cherns [15] principles, their motivation levels using a scale of four items adapted from the intrinsic motivation subscale of Self-Regulation Questionnaire [88] and Stepping Motivation Scale [40] and team innovation levels using the eight-item scale [39]. The items for the measures are listed in Appendix A. All the items used a 9-point Likert scale with anchors of 1 (strongly disagree) and 9 (strongly agree). Some items were reverse coded. Scale items were averaged to create an overall value for the self-organization and motivation constructs.

Results and analyses

The *t*-tests (Table 2) show that the means of agile methods were significantly higher than the means of plan-driven methods on all the nine Cherns [15] principles. Thus Hypotheses 1–9 were supported. Further, the motivation and innovation in teams using agile methods of SD were found to be significantly higher than the members using plan-driven methods of SD, thus supporting Hypotheses 11 and 13. The results of Moderated Hierarchical Multiple regression (MHMR) in Tables 3 and 4 show that, controlling for age, gender, experience and size of the projects, self-organization has a significant and positive impact on work motivation and innovation, thereby supporting Hypotheses 10 and 12 and explaining the higher work motivation and innovation of agile methods of

Table 2. Results of project post-completion survey.

Cherns [15] principles	Agile methods (Mean)	Agile methods (SD)	Plan-driven methods (Mean)	Plan-driven methods (SD)	Difference in means
Minimum Critical Specification	6.21	0.59	4.63	0.44	1.58**
Boundaries	5.91	0.62	5.14	0.67	0.77*
Multi-functionality	6.34	0.43	4.75	0.82	1.59**
Support Congruence	7.45	0.34	6.01	0.65	1.44**
Feedback	6.78	0.67	5.23	0.39	1.55**
Incompletion	7.32	0.72	4.68	0.48	2.64***
Compatibility	6.54	0.55	5.23	0.55	1.31**
Sociotechnical Criterion	7.77	0.39	5.51	0.70	2.26***
Human Values	6.78	0.79	5.11	0.44	1.67**
Work Motivation	6.72	0.86	5.71	0.93	1.01*
Innovation	7.23	1.17	6.14	1.02	1.09*

*$p < .05$; **$p < .01$; ***$p < .001$; S.D. = Standard deviation.

Table 3. MHMR analysis results for innovation.

Step	Variables added in each step	Change in R-square	Regression coefficients
1.	Control variables: Age, Gender, Experience and Size of SD projects	0.145***	2.88***
2.	Main effects: Self-organization	0.094**	1.64***

* $p < .05$; ** $p < .01$; ***$p < .001$.

Table 4. MHMR analysis results for work motivation.

Step	Variables added in each step	Change in R-square	Regression coefficients
1.	Control variables: Age, Gender, Experience and Size of SD projects	0.178***	3.17***
2.	Main effects: Self-organization	0.123***	2.78***

*$p < .05$; **$p < .01$; ***$p < .001$.

SD compared to plan-driven methods. MHMR analysis is a widely recommended method for testing the significance of the increment in criterion variance explained by the main effects after controlling for the variance due to extraneous variables [21, 22, 25],

Before analyzing the results, the factor analysis procedure was carried out using IBM© SPSS© Statistics Version 19. Dimension reduction was performed on the data pertaining to the two measurement scales. All items of a scale (Self-Organization: S1–S9; Work Motivation: W1–W4 and Team Innovation: I1–I8) loaded on the respective factors. Convergent and discriminant validities between scales were evident by the high loadings within scales (>.50) and no cross loadings (>.40) between scales (see Appendix B). The internal reliabilities of the scales used in the study were examined. The alpha reliabilities of .843 and .821 for Self-Organization and Work Motivation, respectively, were greater than .70.

Conclusion

The study makes a number of contributions to the existing body of knowledge in the area of SD methods. First, it develops and tests a measure to assess the level of self-organization in SD projects based on the theoretical foundations of the nine principles of STS design by Cherns [15]. Self-organization is a construct widely recognized as responsible for many salutary psychosocial and work outcomes in projects. The measure was empirically validated and found useful in explaining the higher innovation and work motivation of team members of agile SD projects compared to team members of plan-driven methods of SD found across multiple studies in the past [19, 26, 55, 57, 58, 64].

Further, the study could identify the distinguishing characteristics of agile and plan-driven methods of SD from the perspective of STS work design. "Theoretically comprehending the distinction between agile methods and plan-driven methods is a concern begging for research attention." [24]. These differences can be used for not only characterizing but also making structuring decisions of SD teams along the agile-plan-driven continuum depending on the project objectives and context. Additionally, the measure developed for self-organization could be used for assessing the level of agility in SD projects and to identify areas across the nine dimensions of STS design where improvements can be made.

However, it should be noted that although SD methods are broadly classified into two categories, the agile methods and the Tayloristic methods, within each category there are many different methods, each with their own principles and practices, making comparisons between them confounding. Hence the results only broadly reflect the distinction between agile methods and plan-driven methods on Cherns' [15] nine principles of STS, and team innovation and motivation. The sample size did not permit further statistical analyses of differences within these two major paradigms.

References

[1] The Agile Manifesto. 2001. http://agilemanifesto.org/
[2] Abrahamsson P, Conboy K, Wang X. 2009. Lots done, more to do: The current state of agile systems development research. Eur J Inf Syst. (18):281–284.
[3] Adler PS, Cole RE. 1993. Designed for learning: A tale of two auto plants. Sloan Manage Rev. 85–94.
[4] Ambler SW. 2006. Survey says: Agileworks in practice. Dr. Dobb's Portal Archit Des.
[5] Bailyn L. 1985. Autonomy in the industrial R&D laboratory. Hum Resour Manage. 24(1):129–146.
[6] Batt R. 1999. Work organization, technology, and performance in customer service and sales. Ind Labor Relat Rev 52(4):539–564.
[7] Baumeister R, Leary MR. 1995. The need to belong: Desire for interpersonal attachments as a fundamental human motivation. Psychol Bull. 117(3):497–529.
[8] Baxter G, Sommerville I. 2011. Socio-technical systems: From design methods to systems Engineering. InteractComput23(1):4–17.
[9] Beck K. 1999. Extreme Programming Explained: Embrace Change. 1st edn. Reading, MA: Addison-Wesley Professional.
[10] Beecham S, Baddoo N, Hall T, Robinson H, Sharp H. 2008. Motivation in software engineering: A systematic literature review. Inf Software Technol 50(9):860–878.
[11] Bemer RW. 1969. Position papers for panel discussion: The economics of program production. Information Processing 68, Amsterdam, North-Holland, pp. 1626–1627.
[12] Brooks FP. 1997. No Silver Bullet: Essence and Accidents of Software Engineering. Reprinted in the 1995 edition of The Mythical Man-Month.
[13] Burr R, Cordery JL. 2001. Self-management efficacy as a mediator of the relation between job design and employee motivation. Hum Perform. (14):27–44.
[14] Chau T, Maurer F, Melnik G. 2003. Knowledge sharing: agile methods vs. Tayloristic methods. In: Proceedings of the 12th IEEE International Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE'03), IEEE Computer Society Press, Los Alamitos, CA, USA, pp. 302–307.
[15] Cherns A. 1987. Principles of sociotechnical design revisited. Hum Relat. (40):153–162.
[16] Cho JJ. 2010. An exploratory study on issues and challenges of agile software development with scrum. All Graduate Theses and Dissertations.
[17] Christensen T. 1993. A high-involvement redesign. Qual Prog. 105–108.
[18] Cockburn A. 2004. Crystal clear: A human-powered methodology for small teams. Pearson Education.
[19] Cockburn A, Highsmith J. 2001. Agile software development: The people factor. Comput. 131–133.
[20] Cougar J. 1990. Ensuring creative approaches in information system design. Managerial Decis Econ. 11(2):281–295.
[21] Cohen J. 1978. Partialled products are interactions: Partialled powers are curve components. Psychol Bull. (85):858–866.

[22] Cortina JM. 1993. Interaction, nonlinearity, and multicollinearity: Implications for multiple regression. J Manage. 19(4):915–922.
[23] de Charms R. 1968. Personal Causation. New York, NY: Academic Press.
[24] Dingsøyr T, Nerur S, Balijepally V, Moe NB. 2012. A decade of agile methodologies: Towards explaining agile software development. J Syst Softw. 85(6):1213–1221.
[25] Dunlap WP, Kemery ER. 1987. Failure to detect moderating effects: Is multicollinearity the problem? Psychol Bull. (102):418–420.
[26] Dyba T, Dinsoyr T. 2008. Empirical studies of agile software development: A systematic review. Inf Softw Technol. 50(9–10):833–859.
[27] Emery F. 1993. Epilogue. In: FM van Eijnatten (ed.), The Paradigm that Changed the Work Place. Assen: Van Gorcum.
[28] Eva M. 1994. SSADM version 4: A user's guide, In: D. Ince (ed.), The McGraw-Hill International Series on Software Engineering, 2nd edn. London: McGraw-Hill Book Company.
[29] Fenton-O'Creevy M. 1998. Employee involvement and the middle manager: Evidence from a survey of organizations. J Organ Behav. 19(1):67–84.
[30] Florac WA, Carleton AD. 1999. Measuring the Software Processes: Statistical Control for Software Process Improvement. Reading, MA: Addison Wesley.
[31] Fraser R. 1947. The incidence of neurosis among factory workers. Report No. 90. Industrial Health Research Board. London: HMSO.
[32] Frese M. 1989. Theoretical models of control and health. In: SL Sauter, JJ Hurrell, Jr, CL Cooper (eds.), Job Control and Worker Health. New York: Wiley, pp. 108–128.
[33] Frese M, Kring W, Soose A, Zempel J. 1996. Personal initiative at work: Differences between East and West Germany. Acad Manage J. (39):37–63.
[34] Gamma E, Beck K. 2002. *JUnit*, Testing Resources for Extreme Programming.
[35] Gist ME, Mitchell TR. 1992. Self-efficacy: A theoretical analysis of its determinants and malleability. Acad Manage Rev. 17(2):183–211.
[36] Graham I, Henderson-Sellers B, Younessi H. 1997. The OPEN process specification. In: B Henderson-Sellers (ed.), The OPEN series. Harlow, England: Addison-Wesley.
[37] Guzzo RA, Dickson MW. 1996. Teams in organizations: Recent research on performance and effectiveness. Ann Rev Psychol. (47):307–338.
[38] Hackman JR, Oldham GR. 1976. Motivation through the design of work: Test of a theory. Organ Behav Hum Perform. (16):250–279.
[39] Burpitt WJ, Bigoness WJ. 1997. Leadership and innovation among teams. Small Group Res (28):414–423.
[40] Hayamizu T. 1997. Between intrinsic and extrinsic motivation: Examination of reasons for academic study based on the theory of internalization. Jpn Psychol Res. 39(2):98–108.
[41] Heidenberg J. 2011. Towards Increased Productivity and Quality in Software Development Using Agile, Lean and Collaborative Approaches. Turku: Turku Centre for Computer Science.
[42] Herbst PG. 1962. Autonomous Group Functioning. London: Tavistock.
[43] Highsmith J. 2002. Agile Software Development Ecosystems. Boston, MA: Pearson.
[44] Hislop GW, Lutz MJ, Naveda JF, McCracken WM, Mead NR, Williams LA. 2002. Integrating agile practices into software engineering courses. Comput Sci Educ. 12(3):169–185.
[45] Hoda R. Self-organizing agile teams: A grounded theory, PhD Thesis, Victoria University of Wellington, New Zealand.
[46] Hoegl M, Parboteeah KP. 2006. Autonomy and teamwork in innovative projects. Hum Resour Manage. 45(1):67–79.
[47] Huber VL, Brown KA. 1991. Human resource issues in cellular manufacturing: A sociotechnical analysis. J Oper Manage. (19):138–159.
[48] Huh WT. 2001. Software process improvement: Operations perspectives. In: Management of Engineering and Technology, PICMET'01. Portland International Conference, Vol. 1, pp. 428–429.
[49] Hut J, Molleman E. 1998. Empowerment and team development. Team Perform Manage. 4(2):53–66.
[50] Jacobson I, Booch G, Rumbaugh J, Rumbaugh J, Booch G. 1999. The unified software development process. Reading: Addison-Wesley.
[51] Kajko-Mattsson M, Lewis GA, Siracusa D, Nelson T, Chapin N, Heydt M, Nocks J, Snee H. 2006. Longterm life cycle impact of agile methodologies. In: Proceedings of the 22nd IEEE International Conference on Software Maintenance (Philadelphia, Pennsylvania, USA), IEEE Computer Society, pp. 422–425.
[52] Kanigel R. 1997. The One Best Way: Frederick Winslow Taylor and the Enigma of Efficiency. Viking, New York.
[53] Klein JA. 1989. The human costs of manufacturing reform. Harv Bus Rev. 60–66.
[54] Kohn ML, Schooler C. 1978. The reciprocal effects of the substantive complexity of work on intellectual complexity: A longitudinal assessment. Am J Sociol. (84):24–52.
[55] Layman L, Williams L, Cunningham L. 2004. Exploring extreme programming in context: An industrial case study. Agile Development Conference.
[56] Lobert B, Dologite D. 1994. Measuring creativity of information system ideas: An exploratory investigation, in Proceedings of the Annual Hawaii International Conference on Systems Science, Hawaii, IEEE Computer Society.
[57] Mann C, Maurer F. 2005. A case study on the impact of scrum on overtime and customer satisfaction. Agile Development Conference.
[58] Mannaro K, Melis M, Marchesi M. 2004. Empirical analysis on the satisfaction of IT employees comparing XP practices with other software development methodologies. In: Extreme Programming and Agile.
[59] Martin J, Finkelstein C. 1981. Information Engineering. Vols. 1 and 2. Englewood Cliffs, New Jersey: Prentice Hall.
[60] Mathisen G, Einarsen S. 2004. A review of instruments assessing creative and innovative environments within organisations. Creativity Res J 16(1):119–140.
[61] Maurer F, Melnik G. 2005. What you always wanted to know about agile methods but did not dare to ask. In: ICSE '05: Proceedings of the 27th International Conference on Software Engineering, ACM Press, New York, NY, USA, pp. 731–732.
[62] McAvoy J, Butler T. 2009. The role of project management in ineffective decision making within Agile software development projects. Eur J Inf Syst. 18(4)372–383.
[63] McIlroy MD. 1968. Mass produced software components. Report NATO . on Software EngineeringConference, Garmisch, pp. 138–152.
[64] Melnik G, Maurer F. 2006. Comparative analysis of job satisfaction in agile and non-agile software development teams, XP2006.
[65] Miles RE, Snow CC, Miles G. 2000. TheFuture.org. Long Range Plann 33(3):300–321.
[66] Mirsky J, Nevins A. 1952. The World of Eli Whitney. New York: The Macmillan Company.
[67] Moe, N. B., Aurum, A., and Dybå, T. 2012. Challenges of shared decision-making: A multiple case study of agile software development. Inf Softw Technol 54(8),853–865.
[68] Moe NB, Dingsøyr T, Dybå T. 2009. Overcoming barriers to self-management in software teams. IEEE Softw. 26(6):20–26.
[69] Moe NB, Dingsoyr T, Dyba T. 2008. Understanding self-organizing teams in Agile software development. In: ASWEC 08 (Washington, 2008), IEEE, pp. 76–85.

[70] Morgan G. 1986. Images of Organizations. Beverly Hills: Sage Publications.
[71] Mumford E. 2006. The story of socio-technical design: Reflections on its successes, failures and potential. Inf Syst J. 16(4)317–342.
[72] Elam J, Mead M. 1987. Designing for creativity: Considerations for DSS development. Inf Manage. 13(2):215–222.
[73] Nerur S, Balijepally V. 2007. Theoretical reflections on agile development methodologies. Commun ACM 50(3):79–83.
[74] Nerur S, Mahapatra R, Mangalaraj G. 2005. Challenges of migrating to agile methodologies. Commun ACM 72–78.
[75] Niepcel W, Molleman E. 1998. Work design issues in lean production from a sociotechnical systems perspective: neo-Taylorism or the next step in sociotechnical design? Hum Relat 51(3):259–287.
[76] Nonaka I. 1991. The knowledge-creating company. Harv Bus Rev 69(6):96–104.
[77] Kramer RM. 1999. Trust and distrust in organizations: Emerging perspectives, enduring questions. Ann Rev Psychol 50(1):569–598.
[78] Ogawa S, Piller FT. 2006. Reducing the risks of new product development. MIT Sloan Manage Rev 4(2)65.
[79] Palmer SR, Felsing M. 2001. A Practical Guide to Feature-Driven Development. Upper Saddle River, NJ: Prentice Hall.
[80] Parker SK. 1998. Role breadth self-efficacy: Relationship with work enrichment and other organizational practices. J Appl Psychol. (83):835–852.
[81] Parker SK, Wall T. 1998. Job and Work Design: Organizing Work to Promote Well-Being and Effectiveness, London: Sage.
[82] Parker SK, Wall TD, Jackson PR. 1997. That's not my job': Developing flexible employee work orientations. Acad Manage J. (40):899–929.
[83] Paulk MC, Weber CW, Curtis B, Chrissis MB. 1995. The Capacity Maturity Model: Guidelines for Improving the Software Process. Boston, MA: Addison Wesley.
[84] Poppendieck M, Poppendieck T. 2003. Lean Software Development: An Agile Toolkit. Boston, MA: Addison-Wesley Professional.
[85] Roberts JA, Hann IH, Slaughter SA. 2006. Understanding the motivations, participation, and performance of open source software developers: A longitudinal study of the Apache projects. Manage Sci. 52(7):984–999.
[86] Royce WW. 1970. Managing the development of large software systems. In: Proceedings of IEEE WESCON (26, 8).
[87] Ryan RM, Deci EL. 2000. Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. Am Psychol 55(1):68–78.
[88] Ryan RM, Connell JP. 1989. Perceived locus of causality and internalization: examining reasons for acting in two domains. J Personality Soc Psychol 57(5):749.
[89] Sampler J, Galleta D. 1991. Individual and organisational changes necessary for the application of creativity techniques in the development of information systems. In: Proceedings of the 24th Annual Hawaii International Conference on System Sciences, Hawaii, IEEE Society Press.
[90] Scrum Alliance. 2008. World Wide Web electronic publication, http://www.scrumalliance.org/view/scrum_framework.
[91] Schwaber K, Sutherland J. 2007. Scrum. *URL*: http://wwwscrumalliance. org/system/resource/file/275/whatIsScrum. pdf,[Sta nd: 03.03. 2013].
[92] Shingo S. 1986. Zero Quality Control: Source Inspection and the Poka-Yoke System. Cambridge, MA: Productivity Press.
[93] Stapleton J. 1997. DSDM, Dynamic Systems Development Method: The Method in Practice. Cambridge: Cambridge University Press.
[94] Takeuchi H, Nonaka I. 1986. The new product development game. Harv Bus Rev. (64)137–146.
[95] Tata J, Prasad S. 2004. Team self-management, organizational structure, and judgments of team effectiveness. J Managerial Issues 16(2)248–265.
[96] Taylor FW. 1911. The Principles of Scientific Management. New York: Harper and Bros.
[97] Gallivan M. 2003. The influence of software developer's creative style on their attitudes to and assimilation of a software process innovation. Inf Manage 40(1):443–465.
[98] Trist E. 1981. The Evolution of Socio-Technical Systems: A Conceptual Framework and an Action Research Program. Ontario: Ontario Ministry of Labor.
[99] Trist E, Bamforthe KW. 1951. Some social and psychological consequences of the Longwall Method of coal-getting. Hum Relat. (4):6–24.
[100] Turk D, Robert F, Rumpe B. 2005. Assumptions underlying agile software-development processes. JDM. 16(4):62–87.
[101] Walker CR, Guest R. 1952. The Man on the Assembly Line. Cambridge, MA: Harvard University Press.
[102] White RW. Ego and Reality in Psychoanalytic Theory. New York: International Universities Press.
[103] Speier C, Frese M. 1997. Generalized self-efficacy as a mediator and moderator between control and complexity at work and personal initiative: A longitudinal field study in East Germany. Hum Perform (10):171–192.

Appendix A

Table A1. Measures used in the study.

Measures and items
Self-organization
The job provides substantial freedom, independence and discretion to the team members in setting team goals, in scheduling work and in determining the procedures to be used in crying it out
The job involves significant interaction with employees and groups outside of the team (reverse coded)
The job requires significant skill variety from its team members in executing tasks
The job involves employee training and coaching on a variety of skills required to achieve team goals
Managers and coworkers let me know how well I am doing on my job
The job involves work that can be completely planned upfront (reverse coded)
All team members participate in the design process
The job involves deviation from expected standards to be detected and corrected at the source
My organization is genuinely interested in my well-being
Work motivation
I found the work in the project to be interesting
I am glad to have worked on the project
Compared to other similar things I have done working on the project was truly enjoyable
I was so totally immersed while working on the project that it made me forget my problems
Innovativeness
Using skills they already possess, this team learns new ways to apply those skills to develop new solutions to business problems.
The team seeks out information about new technologies and trends in software
This team identifies and develops skills that can improve their ability to serve existing business needs.
This team identifies and develops skills that can help serve new business needs.
This team learns new ways to apply their knowledge of familiar patterns and techniques to develop new and unusual solutions to familiar, routine problems.
This team learns new ways to apply their knowledge of familiar products and techniques to develop new and unusual solutions to familiar, routine problems.
This team seeks out and acquires information that may be useful in exploring and developing multiple solutions to problems.
This team seeks out and acquires knowledge that may be useful in satisfying needs unforeseen by the business user.

Table A2. Results of factor analyses.

	Factors		
Items	1	2	2
S1	**0.939**	0.015	0.015
S2	**0.904**	0.016	0.016
S3	**0.918**	0.018	0.018
S4	**0.789**	−0.026	−0.026
S5	**0.849**	0.003	0.003
S6	**0.841**	−0.025	0.092
S7	**0.879**	0.011	0.275
S8	**0.883**	0.121	0.004
S9	**0.840**	0.129	0.080
M1	0.025	**0.881**	0.113
M2	−0.082	**0.856**	0.015
M3	0.085	**0.836**	0.126
M4	0.016	**0.844**	0.092
I1	0.037	0.203	**0.727**
I2	0.030	0.215	**0.875**
I3	−0.097	0.050	**0.853**
I4	−0.014	0.112	**0.837**
I5	0.036	0.150	**0.857**
I6	0.001	0.067	**0.868**
I7	−0.019	0.022	**0.829**
18	0.016	0.146	**0.786**